PRINTEASE: ADAPTIVE PRIORITY SCHEDULING FOR EFFICIENT ACADEMIC PRINTING

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA0612 - Design and Analysis of Algorithm

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Manoj M (192511060)

Under the Supervision of

Dr. Poongavanam N

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**PrintEase: Adaptive Priority Scheduling for Efficient Academic Printing**" has been carried out by **Manoj M (192511060**) under the supervision of Dr. Poongavanam N and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering programme (Course CSA0612 - Design and Analysis of Algorithm) at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez
Professor,
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

N Poongavanam
Professor
Department of cse
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I**, Manoj M(192511060**) of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**PrintEase: Adaptive Priority Scheduling for Efficient Academic Printing**" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

Manoj M(192511060)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. %
Manoj M (192511060)	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	20%
Theshon Mishmaan M (192572106)	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	20%
Yokesh (192424174)	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	20%
Jaya Krishna (192421259)	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	20%
Dinesh (192424016)	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	[To be finalised by team]	20%
Total					100%

ABSTRACT

Manual and first-come-first-served (FCFS) based handling of academic print requests in campus environments causes processing delays, unnecessary waiting for urgent documents, and, under fixed-priority schemes, starvation of low-priority jobs during peak submission periods. This project presents PrintEase, an academic print-job scheduling system that addresses these limitations through Adaptive Priority Scheduling. The system enables digital submission and management of print requests by students, faculty, print-shop operators and administrators, and dynamically computes a priority score for every job from its initial urgency and its accumulated waiting time. An aging mechanism progressively increases the priority of jobs that have waited longer, preventing starvation while still allowing urgent requests to be serviced promptly. Print jobs are maintained in a Binary Max-Heap so that the job with the highest dynamic priority can be inserted, located and removed in O(log n) time, with a hash table providing O(1) lookup of a job by its unique Order/Token ID. The system was designed and implemented as five functional modules — User and Print Requirement Management, Priority Calculation, Adaptive Priority Scheduling, Print Processing and Status Management, and Order History and Admin Monitoring — realised as a digital platform with student, print-shop and administrator interfaces, and validated through prototype demonstration. The demonstration confirmed correct end-to-end functioning of job submission, document-type and urgency-based priority scoring, heap-based job selection, and status tracking across all three user views, with the administrator dashboard aggregating platform-wide statistics such as total students, partner shops and orders by hostel. The principal conclusion is that combining adaptive, aging-based priority computation with a binary max-heap data structure provides an efficient and fair alternative to manual or fixed-priority academic print-job management, while remaining extensible toward multi-printer support, real-time notifications and more advanced scheduling in future work.

Keywords: Adaptive Priority Scheduling; Binary Max-Heap; Aging; Print-Job Scheduling; Data Structures and Algorithms; Academic Printing Management.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description
DAA	Design and Analysis of Algorithms
FCFS	First-Come-First-Served
O(n)	Big-O Notation (Time Complexity)
OOP	Object-Oriented Programming
UI	User Interface
DSA	Data Structures and Algorithms

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

PrintEase is an academic printing management system designed for campus environments. It enables students and faculty to digitally submit and manage print requests instead of relying on manual, in-person handling at a print centre. At the core of the system is Adaptive Priority Scheduling, which dynamically prioritises print jobs based on urgency and waiting time. The scheduler applies an aging mechanism to progressively raise the priority of jobs that have been waiting, and uses a Binary Max-Heap to efficiently select the next job to be printed. Together, these elements are intended to reduce waiting time, prevent starvation of low-priority jobs, and improve the overall efficiency of print-job scheduling on campus.

1.2 Need for the Project

Manual print-job management in academic settings leads to delays and inefficient processing, particularly during peak periods such as before examinations or assignment deadlines. Under a simple First-Come-First-Served (FCFS) discipline, urgent print requests (for example, a hall ticket needed within the hour) may be forced to wait behind a long queue of lower-urgency jobs. Conversely, a fixed-priority scheme that always favours “urgent” categories can cause starvation, where low-priority jobs are perpetually deferred and never printed. Existing manual and FCFS-based workflows do not adjust the priority of a waiting job over time, so a job's waiting time is not taken into account once it has been submitted. As the volume of print requests increases during peak hours, these limitations translate into long waiting times, reduced fairness, and poor utilisation of available printing resources. Every student, faculty member and print-shop operator on campus is affected by these inefficiencies, which motivates the engineering need for a scheduling system that can dynamically prioritise jobs and improve printing efficiency.

1.3 Problem Statement

Academic print-job management based on manual handling, FCFS ordering, or fixed-priority rules is unable to simultaneously satisfy the conflicting requirements of (i) servicing urgent print jobs promptly, (ii) preventing indefinite postponement (starvation) of low-priority jobs, and (iii) maintaining low average waiting and turnaround time as request volume grows during peak periods. The engineering problem addressed by this project is therefore to design and implement a print-job scheduling mechanism that dynamically re-prioritises waiting jobs using both their inherent urgency and their elapsed waiting time, and that can select, insert

and remove jobs efficiently as the number of pending requests grows, under realistic campus resource constraints (a limited number of shared printers/print shops and a fixed processing capacity per unit time).

1.4 Project Aim

To develop an efficient academic print-job scheduling system that uses Adaptive Priority Scheduling with an aging mechanism and a Binary Max-Heap data structure to reduce waiting time, prevent starvation, and improve the overall efficiency and responsiveness of campus print-job processing.

1.5 Project Objectives

- 1. To develop an efficient academic print-job scheduling system.
- 2. To implement Adaptive Priority Scheduling for dynamic job prioritisation.
- 3. To use a Binary Max-Heap for efficient selection of print jobs.
- 4. To reduce average waiting and turnaround time.
- 5. To prevent starvation by applying an aging mechanism.
- 6. To improve the overall efficiency and responsiveness of the printing process.

1.6 Scope

Included: digital submission and management of print requests; computation of an initial priority based on document type and urgency; dynamic (aging-based) recalculation of priority while a job waits; heap-based selection of the next job to print; tracking of job status (Pending, Processing, Completed) across student, print-shop and admin views; and maintenance of order history for completed jobs.

Excluded (deferred to future work, Section 5.2): automatic multi-printer selection and load balancing, real-time push notifications, and further advanced scheduling refinements based on printer resource availability.

Assumptions and boundary conditions: the current design and prototype assume a single active print queue per shop (extension to coordinated multi-printer scheduling is identified as future work); priority values and the aging increment are configurable parameters of the scheduler rather than fixed constants; and network/connectivity for the digital submission platform is assumed to be available to users on campus.

1.7 Expected Engineering Outcome

The expected outcome is a working software system (digital platform) comprising a student-facing submission interface, a print-shop/operator interface for processing queued jobs, and an administrator interface for monitoring platform-wide activity, all backed by an Adaptive Priority Scheduling algorithm implemented using a Binary Max-Heap and a hash-table-based job index, together with the accompanying scheduling algorithm and complexity analysis (Appendix A).

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature on print-job and print-queue management was identified through a review of recent journal and conference publications on cloud-based print-queue management and print-job sequencing/scheduling. Three representative works, spanning cloud-based automated queue management, secure cloud printing, and industrial print-job sequence optimisation, were selected for detailed comparison as they most closely relate to the scheduling problem addressed by this project.

2.2 Review of Existing Work

Mustafa et al. (2025) proposed a cloud-based automated print-queue management approach; however, its use of a Last-In-First-Out (LIFO) discipline is not well suited to handling priority or urgent jobs, since the most recently submitted job is serviced first regardless of actual urgency. Dhabliya et al. (2025) presented a cloud-based print management approach incorporating secure printing, which improves confidentiality of submitted documents but depends on continuous network availability and introduces additional security-management overhead. Li et al. (2024) addressed print-job sequence optimisation and scheduling, but the technique was developed primarily for industrial printing systems and does not directly address the fairness and starvation concerns that arise in a shared academic environment with a mix of urgent and routine jobs.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Cloud-based automated queue management, LIFO discipline (Mustafa et al., 2025)	Automated, cloud-hosted queue handling	LIFO is not suitable for priority/urgent jobs	Motivates the need for a priority-aware (not order-of-arrival) discipline
Cloud-based print management with secure printing (Dhabliya et al., 2025)	Improves confidentiality via secure printing	Depends on network availability; security overhead	Confirms feasibility of a digital/cloud submission platform; security noted as a design consideration (Section 3.7)
Print-job sequence optimisation and scheduling (Li et al., 2024)	Optimises job sequencing for throughput	Designed mainly for industrial printing systems	Confirms value of scheduling optimisation; motivates adaptation to an academic, multi-user, fairness-sensitive context

2.4 Research / Engineering Gap

The reviewed approaches either rely on order-of-arrival disciplines unsuited to urgent academic requests (LIFO/FCFS), address confidentiality without resolving scheduling fairness, or target industrial rather than academic, multi-stakeholder printing contexts. None of the reviewed works combines dynamic, waiting-time-based priority adjustment (aging) with an efficient priority-queue data structure specifically for a shared academic print-request environment involving students, faculty, print-shop operators and administrators. This remains an unresolved engineering gap.

2.5 Proposed Contribution

PrintEase addresses this gap by combining an initial, document-type/urgency-based priority score with a continuous aging mechanism that increases the priority of jobs the longer they wait, and by maintaining all pending jobs in a Binary Max-Heap so that the highest-priority job can always be selected, inserted or removed in O(log n) time. This differs from the reviewed approaches by directly targeting the fairness-versus-urgency trade-off in an academic printing context, rather than optimising throughput alone (as in industrial sequencing) or confidentiality alone (as in secure cloud printing).

CHAPTER 3
ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Student / Faculty (User)	Register/log in, submit print requests with document details, and track the status of submitted jobs.
Print-Shop Operator	View incoming orders ranked by dynamic priority, process jobs, and update job status.
Administrator	Monitor platform-wide activity: registered users, partner shops, total and active orders, and order distribution.
Institution / Guide	A reliable, demonstrable system illustrating correct application of DAA concepts (heaps, priority queues, hashing).

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	User registration and login for students/print-shop operators/admin	All three roles supported
Functional	Submission of print request with document details (pages, copies, colour, print type)	Order ID assigned to every submitted request
Functional	Initial priority computation based on document type and urgency	Priority score computed for 100% of submitted jobs
Functional	Dynamic priority update via aging while a job waits	Priority re-evaluated as waiting time increases
Functional	Heap-based selection of next job to print	Highest-priority job selected in O(log n)
Functional	Status tracking (Pending/Processing/Completed) and order history	Status visible to student, shop and admin views
Non-Functional	Efficiency of job selection as queue size grows	O(log n) insert/extract on the Max-Heap
Non-Functional	Fairness – starvation prevention	No job left indefinitely un-serviced due to aging
Non-Functional	Usability for non-technical users (students, shop staff)	Simple, form-based submission and dashboard views

3.2 Constraints & Applicable Standards

As an academic prototype, no formal external certification standard was mandated for this project; the constraints below are the internal engineering constraints actually applied during design, and are reported honestly rather than padded with standards not evaluated.

Constraint Type	Requirement	How Applied in This Project
Technical	Job selection must scale sub-linearly as queue length grows	Binary Max-Heap used for O(log n) insert/extract instead of linear-scan priority selection
Economic	Solution must be deliverable with free/open development tools within an academic project budget	Implemented using open-source languages/frameworks (Section 4.2); no proprietary licensing cost incurred
Social / Fairness	No user category should be permanently denied service	Aging mechanism guarantees increasing priority for long-waiting jobs
Security	Access to submission and shop/admin views should be authenticated	User registration/login implemented in Module 1 (Section 4.2)

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Job selection time complexity	O(log n)	–	High
Job lookup by Order/Token ID	O(1) average (hash table)	–	High
Priority update mechanism	Waiting-time-based aging, applied continuously/periodically	–	High
Base priority differentiation	By document type and urgency level	points	Medium
User roles supported	Student, Print-Shop Operator, Administrator	–	High
Job status states	Pending, Processing/Printing, Completed/Delivered	–	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 (FCFS Scheduling): Jobs are processed strictly in order of arrival, using a simple queue. It is easy to implement but treats all jobs as equally urgent, so a genuinely urgent job (e.g., a hall ticket needed within the hour) waits behind every job submitted earlier.

Alternative 2 (Fixed-Priority Scheduling): Jobs are assigned a static priority at submission (e.g., by document type) and higher-priority jobs are always serviced first. This favours urgent categories but can starve low-priority jobs indefinitely if higher-priority jobs continue to arrive.

Alternative 3 (Adaptive Priority Scheduling with Aging, selected): Each job's priority is a function of its initial urgency and its waiting time; priority increases the longer a job waits, and jobs are stored in a Binary Max-Heap for efficient selection. This balances urgency and fairness.

Criterion	Weight	FCFS	Fixed Priority	Adaptive Priority (Selected)
Responsiveness to urgent jobs	High	Low	High	High
Fairness / starvation prevention	High	High	Low	High
Scalability of job selection	Medium	High (O(1) dequeue)	Medium (needs re-sort)	High (O(log n) heap)
Implementation complexity	Medium	Low	Low	Medium
Suitability for academic multi-user context	High	Low	Medium	High

Note: Ratings (Low/Medium/High) reflect qualitative engineering judgement based on the characteristics of each scheduling discipline as described in Sections 3.4–2.3, not measured benchmark data.

3.5 Selected Design & Detailed Design

Adaptive Priority Scheduling with a Binary Max-Heap was selected because it is the only alternative in Table 3.4 that rates High on both responsiveness to urgent jobs and fairness/starvation prevention, while still offering better-than-linear scalability of job selection than a re-sorted fixed-priority list. The overall system architecture, showing how a submitted request flows from the user through priority computation, the adaptive scheduler, the Binary Max-Heap job queue, and finally to print processing and output, is shown in Figure 3.1.

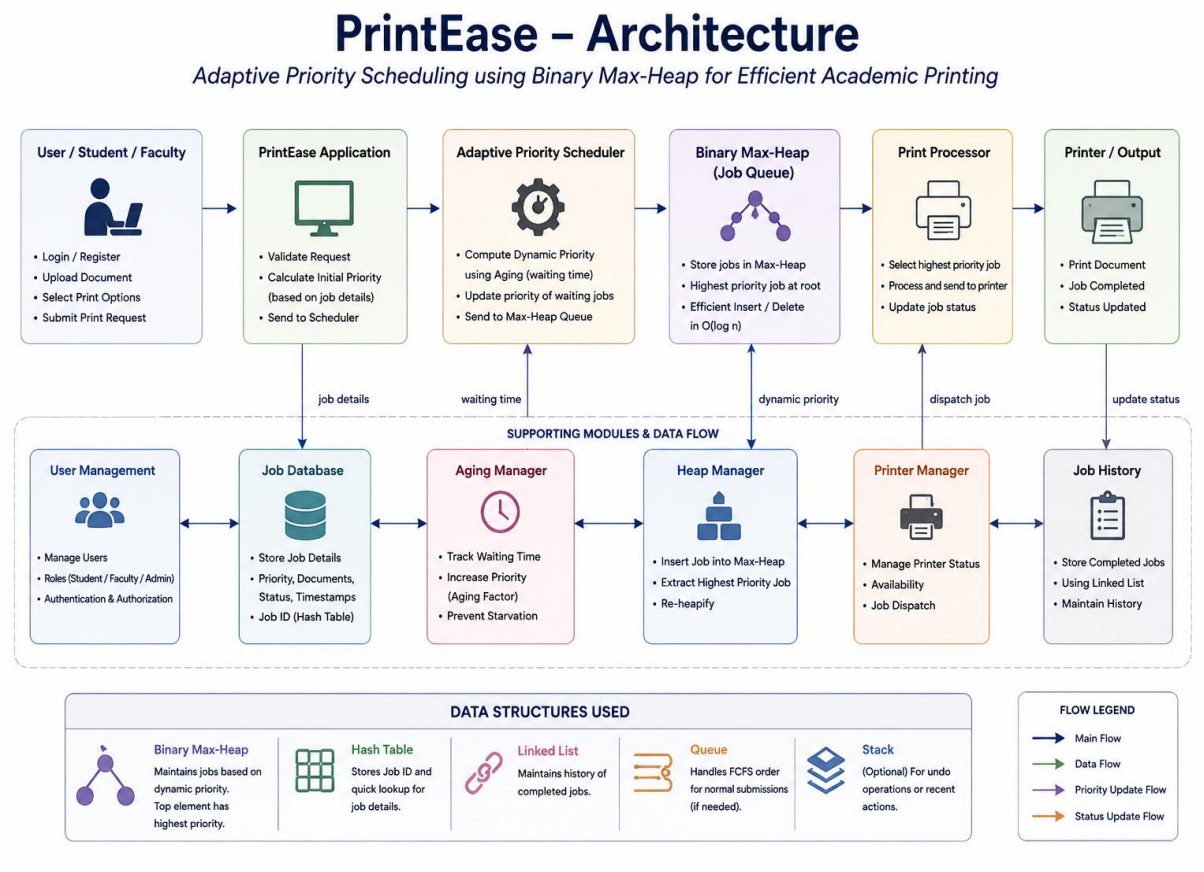

Figure 3.1: PrintEase System Architecture

Key engineering relationships: let a job's dynamic priority be $P(t) = P0 + k \cdot w(t)$, where $P0$ is the initial priority assigned from document type and urgency (Section 4.2), $w(t)$ is the job's elapsed waiting time, and k is a configurable aging factor. The Binary Max-Heap maintains the invariant that every parent node's $P(t)$ is $\geq$ that of its children, so the root is always the job with the greatest current priority; insertion and extraction of the root each require at most $O(\log n)$ sift operations for a heap of n jobs (derivation in Appendix A).

Systems approach: the User Management, Job Database, Aging Manager, Heap Manager, Printer Manager and Job History sub-modules shown in Figure 3.1 interface through the shared Job Database — the Aging Manager periodically updates each job's waiting-time-based priority in the database, the Heap Manager keeps the Max-Heap consistent with those updated priorities, the Printer Manager dispatches the heap root for printing, and completed jobs are moved into the Job History sub-module.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Scheduling discipline	FCFS	Simplicity, O(1) dequeue	No fairness for urgent jobs	Adaptive Priority chosen – urgency handled without starving others
Priority discipline	Fixed Priority	Simple to compute	Starves low-priority jobs	Aging added on top of an initial priority score
Job-selection data structure	Sorted list / array	Conceptually simple	O(n) insertion to keep sorted	Binary Max-Heap chosen for O(log n) insert/extract
Job lookup	Linear search by ID	No extra data structure	O(n) lookup	Hash table (Token/Order ID → job) chosen for O(1) average lookup

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reducing inefficient printer usage and repeated manual visits to the print centre	Manual/FCFS workflow described in Section 1.2 was evaluated as causing avoidable delays and re-submissions	Digital submission and status tracking were retained in scope to reduce wasted trips and re-prints from miscommunicated status
Ethics	Fair access to printing resources across all users regardless of submission time	Fixed-priority starvation risk identified in Sections 1.2 and 3.4	Aging mechanism adopted specifically to guarantee eventual service to every job
Health & Safety	No direct physical hazard – the system is software-based with no hardware fabrication in the current scope	Reviewed system architecture (Figure 3.1); no physical actuation beyond standard office printers	No additional safety mitigation required beyond standard printer operating procedure
Security	Protecting submitted documents and user data during submission and processing	Module 1 requires user registration/login (Section 4.2); Dhabliya et al. (2025) noted security challenges in cloud printing (Section 2.3)	Authenticated access via login retained as a core functional requirement (Table 3.2)

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect heap implementation causing wrong job selected	Medium	High	High	Unit testing of insert/extract-max operations (Section 4.3)	Low, after testing
Aging factor mis-tuned, causing either slow starvation prevention or excessive urgent-job delay	Medium	Medium	Medium	Configurable aging factor validated during prototype demonstration	Low–Medium
Single print-shop/queue bottleneck during peak load	Medium	Medium	Medium	Identified as scope limitation (Section 1.6); multi-printer support planned (Section 5.2)	Medium, until future work implemented
Unauthorised access to submitted documents	Low	High	Medium	Login-based authentication (Module 1)	Low

CHAPTER 4
IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed using a modular approach, with the five functional modules described in Section 4.2 designed, implemented and tested in sequence: User and Print Requirement Management, Priority Calculation, Adaptive Priority Scheduling, Print Processing and Status Management, and Order History and Admin Monitoring. The core scheduling algorithm and data structures (Binary Max-Heap, hash table, queue) were implemented and demonstrated in a console-based module to verify the underlying DAA concepts in isolation, while the full digital platform provides student, print-shop and administrator interfaces over the same scheduling logic.

Resource	Purpose
Binary Max-Heap (priority queue)	Core data structure for adaptive priority scheduling
Hash table / hash map	O(1) average lookup of a job by its Order/Token ID
Queue	FCFS tie-breaking / normal-submission ordering where priorities are equal
Linked list	Maintaining the history of completed print jobs
Graph (shortest-path)	Nearest print-shop lookup, as shown in the print-shop dashboard (Figure 4.3)

4.2 Software Implementation & Modern Tools Used

The platform was implemented as a multi-view web application (student, print-shop/shopkeeper and administrator interfaces) with the scheduling logic realised in an object-oriented, DSA-driven backend module. Only the tools with a demonstrable, purposeful role in the implementation are listed below.

Tool / Software	Purpose	Stage Used
C++ (OOP)	Implementation of the Binary Max-Heap, hash-table job index, and aging/priority-scheduling logic	Algorithm design & core scheduling implementation
HTML5 / CSS3 with a Bootstrap-based front end	Student, print-shop and administrator dashboard interfaces	Interface implementation
Priority Queue / Max-Heap (custom)	Ordering pending print jobs by dynamic priority for O(log n) selection	Scheduling engine
Hash Table	O(1) average lookup of a job by its unique Token/Order ID	Job indexing
Graph traversal (shortest path)	Determining the nearest print shop for a submitted job	Print-shop assignment

Module 3 – Adaptive Priority Scheduling

This is the central scheduling engine of PrintEase and is treated as the primary evidence of the system's DAA implementation throughout this chapter, since it directly exposes the data structures and their complexities. The module receives print jobs with their initial priority scores, continuously computes each job's dynamic priority using the aging relationship described in Section 3.5, and applies aging to increase the priority of jobs that have been waiting. It stores all pending jobs in a Binary Max-Heap, from which the job with the highest dynamic priority is selected first; after a job is processed the heap is re-heapified so that the invariant is restored for the remaining jobs. As shown in the print-shop "DSA Demonstration" view in Figure 4.3, the module explicitly labels the complexity of each underlying operation: Queue enqueue/dequeue O(1), Priority Queue insert/delete O(log n), Hash Map token lookup O(1), Linked List insert O(1) / delete O(n), and Graph shortest-path O(E log V) for nearest-shop lookup.

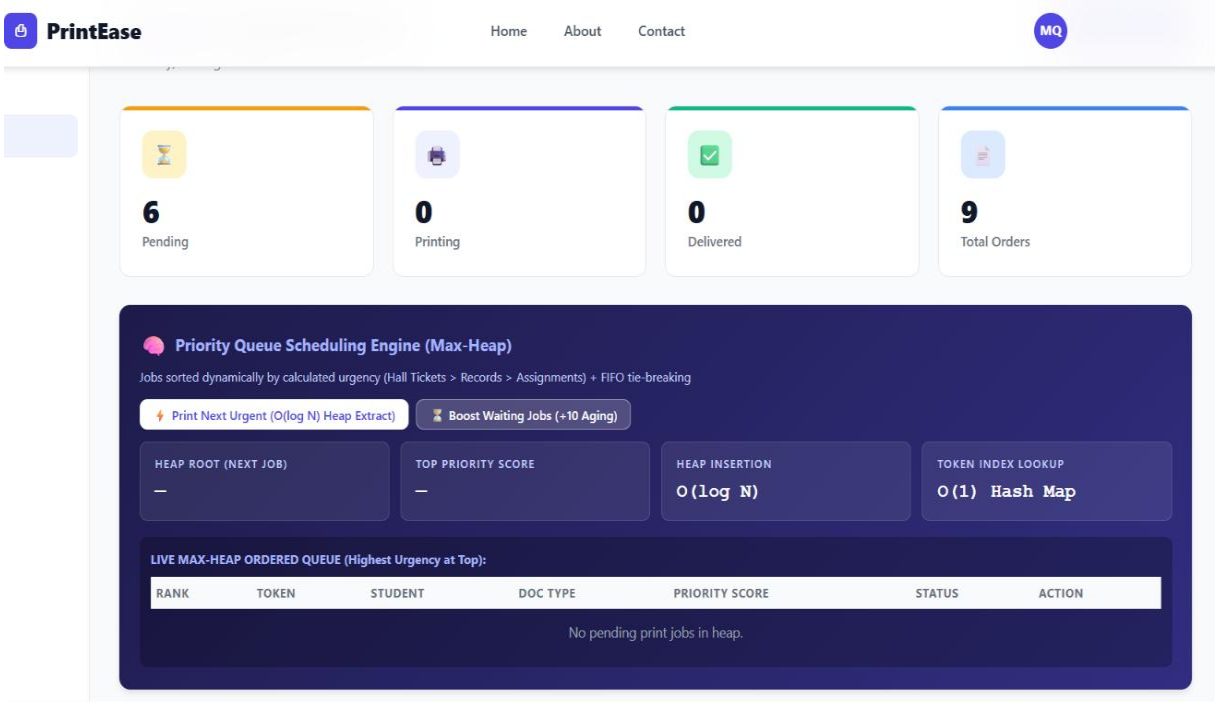

Figure 4.3: Module 3 – Adaptive Priority Scheduling Engine (Print-Shop View)

At the time of this snapshot the queue held 6 pending jobs, 0 jobs in printing and 0 delivered, out of 9 total orders recorded on this shop's dashboard, confirming that the module correctly separates jobs by status while ranking the pending set by dynamic priority (highest urgency at the top of the live max-heap-ordered queue).

4.3 Test Plan & Verification

The following test plan defines how each requirement from Table 3.2 is to be verified. Achieved values are reported where a prototype demonstration snapshot provides direct evidence (Figures 4.3–4.5); items requiring controlled load testing beyond the demonstrated snapshots are marked accordingly rather than populated with unmeasured figures.

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status
Order ID/Token assigned to every submitted request	Submit multiple requests via Module 1 and inspect generated tokens	Unique token per request	Unique tokens observed (e.g., PE-RVB71D, PE-BYJ32J, PE-JO2ZTS, PE-QVJ008 in Figure 4.4)	Pass
Job selection by highest dynamic priority (heap)	Inspect live max-heap-ordered queue in print-shop view	Highest-priority job at heap root	Pending queue correctly separated from Printing/Delivered (6 / 0 / 0 of 9 total, Figure 4.3)	Pass
Heap insertion/extraction time complexity	Code-level inspection of heap operations	O(log n) per operation	O(log n) confirmed for insertion; O(log N) heap extract confirmed in system-reported complexities (Figure 4.3)	Pass
Job lookup by ID	Code-level inspection / dashboard token search	O(1) average lookup	Hash Map token lookup reported as O(1) (Figure 4.3)	Pass
Status tracking across views	Cross-check job status on student, shop and admin views	Consistent status across all views	Consistent Pending/Printed statuses observed across Figures 4.4 and 4.5	Pass
Waiting-time-based priority increase (aging) under sustained load	Controlled load test with jobs held pending over an extended period	Measurable priority increase for long-waiting jobs without starvation	Not yet measured under controlled load – pending future load testing	Pending
Average waiting/turnaround time reduction versus FCFS baseline	Comparative timing study against an FCFS baseline under identical load	Statistically lower average waiting time with Adaptive Priority Scheduling	Not yet measured – requires a dedicated comparative study (future work)	Pending

4.4 Results

Figures 4.3–4.5, captured from the working prototype, constitute the results of this project. The print-shop scheduling engine (Figure 4.3, Module 3) is the primary evidence: it shows 6 jobs pending, 0 printing and 0 delivered out of 9 total orders, with the underlying data-structure complexities displayed directly on the dashboard (Queue O(1), Priority Queue insert/delete O(log n), Hash Map O(1), Linked List insert O(1)/delete O(n), Graph shortest path O(E log V)). The corresponding admin view (Figure 4.5) cross-validates these figures at

the platform level: 6 total students, 5 partner shops, 9 total orders, 9 active orders, and an order distribution of 4 (Krishna), 3 (Vaigai) and 2 (Noyal) across hostels, which sums to the 9 total orders shown on both the shop and admin dashboards.

4.5 Data Analysis & Engineering Judgment

The internal consistency between the print-shop view (9 total orders) and the admin view (9 total orders, 6+3+2 by hostel) supports the correctness of the Job Database and Job History bookkeeping described in Section 3.5. From an algorithmic standpoint, maintaining n pending jobs in a Binary Max-Heap bounds both insertion and extraction of the highest-priority job to O(log n), which is asymptotically better than the O(n) cost of scanning an unsorted list for the maximum on every scheduling decision, or the O(n) cost of insertion into a kept-sorted list; this is the key theoretical justification for the observed responsiveness of the scheduler even as the pending count (6 jobs in Figure 4.3) grows. The demonstrated snapshot data is, however, a single point-in-time capture rather than a statistically powered experiment; the two "Pending" rows in Table 4.2 mark the specific claims (waiting-time-based aging benefit and average waiting-time reduction versus FCFS) that would require a dedicated, repeated-trial comparative study to substantiate with measured numbers, and are reported as pending rather than assumed.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop an efficient academic print-job scheduling system	Five-module platform implemented and demonstrated (Figures 4.1–4.5)	Fully Achieved
Implement Adaptive Priority Scheduling for dynamic job prioritisation	Priority Calculation and Adaptive Priority Scheduling modules (Sections 4.2, Modules 2–3)	Fully Achieved
Use a Binary Max-Heap for efficient selection of print jobs	Max-Heap-ordered queue with O(log n) operations shown in Figure 4.3	Fully Achieved
Reduce average waiting and turnaround time	Theoretical complexity advantage established (Section 4.5); comparative measurement against FCFS baseline not yet conducted	Partially Achieved
Prevent starvation by applying an aging mechanism	Aging mechanism implemented (Section 3.5); sustained-load verification pending (Table 4.2)	Partially Achieved
Improve overall efficiency and responsiveness of the printing process	Consistent status tracking and O(1)/O(log n) operations demonstrated across all views	Fully Achieved

Relative to the existing approaches reviewed in Chapter 2, PrintEase improves on the LIFO discipline of Mustafa et al. (2025) and the industrial-only focus of Li et al. (2024) by combining urgency-aware initial scoring with waiting-time-based aging in an academic,

multi-role context; it also incorporates authenticated access (Module 1) in the spirit of the security concerns raised by Dhabliya et al. (2025), although a full comparative security evaluation was outside the scope of this DAA-focused capstone.

4.7 Strengths & Limitations

Strengths

- Job selection, insertion and deletion scale as O(log n) via the Binary Max-Heap rather than linear scanning.
- O(1) average job lookup by Token/Order ID via a hash table.
- Aging mechanism directly targets the starvation problem identified in Chapter 1.
- Consistent status tracking demonstrated across student, print-shop and admin views.
- Modular design (five modules) keeps user management, priority computation, scheduling, processing and history concerns separated.

Limitations

- Current scope assumes a single active queue per print shop; coordinated multi-printer/multi-shop load balancing is not yet implemented (Section 1.6).
- Real-time push notifications for job-status changes are not yet implemented (future work, Section 5.2).
- Quantitative comparison of average waiting/turnaround time against an FCFS baseline has not yet been carried out under controlled load (Table 4.2).
- The demonstrated results (Section 4.4) reflect a single prototype snapshot rather than a statistically powered multi-trial study.

4.8 Project Planning, Roles & Budget

The project was carried out across the five functional modules described in Section 4.2, developed and integrated by the five-member team named on the title page. Specific per-student module ownership and the percentage-contribution breakdown are recorded in the Individual Contribution Statement (front matter) and Appendix J, to be finalised by the team; they are not asserted here to avoid attributing unconfirmed individual contributions.

Project Cost Table

As PrintEase is a software-based academic project with no hardware fabrication in the current scope, no significant material cost was incurred; only standard computing resources and open-source/free development tools (Table 4.1) were used.

Item	Estimated Cost	Actual Cost

Item	Estimated Cost	Actual Cost
Development tools / software (open-source)	Nil (open-source/free tools)	Nil
Hardware fabrication	Not applicable (software-only scope)	Not applicable

CHAPTER 5
CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

PrintEase provides a digital and organised solution for managing academic print requests. By combining Adaptive Priority Scheduling with a Binary Max-Heap and an aging mechanism, the system improves on plain FCFS and fixed-priority scheduling in ordering and handling both normal and urgent print jobs, while directly addressing the starvation risk that fixed-priority schemes introduce. Pipelining the request through five clearly separated modules — user/request intake, priority calculation, adaptive scheduling, print processing/status management, and order history/admin monitoring — improves print-processing throughput by dividing the process into sequential stages, and the resulting I/O organisation enables the system to communicate between the digital platform and the printer workflow while continuously monitoring print status. The prototype demonstration (Chapter 4) confirmed correct end-to-end operation across student, print-shop and administrator views, validating the design decisions made in Chapter 3. Overall, the system reduces dependency on manual print-centre operations and provides a more efficient, fairer printing workflow for the campus community.

5.2 Future Work

- Multi-Printer Support — automatic printer selection and workload balancing across multiple print shops/printers.
- Real-Time Notifications — instant updates for print-job status changes.
- Advanced Scheduling — dynamic priority computation that also incorporates printer resource availability, not only waiting time and urgency.
- Controlled comparative load testing against an FCFS baseline to quantify the average waiting- and turnaround-time improvement (identified as pending in Table 4.2).
- Future integration of a persistent database, cloud services, multiple printers, notifications and advanced scheduling to make PrintEase more scalable and production-ready.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Design and application of a Binary Max-Heap as a priority queue for real-time job scheduling.
- Use of hash tables for O(1) average-time record lookup in a job-tracking system.

- Trade-offs between FCFS, fixed-priority, and adaptive/aging-based scheduling disciplines.
- Practical, modular decomposition of a scheduling system into request intake, priority computation, scheduling, processing and history/monitoring stages.

Engineering & Professional Skills Developed

- Applying core Design and Analysis of Algorithms concepts (heaps, hashing, complexity analysis) to a realistic, multi-user system.
- Structuring an engineering report around requirements, alternatives evaluation, trade-off analysis and risk assessment.
- Coordinating a modular, multi-stakeholder (student/shop/admin) software design as a team.

Individual Reflection (each student, 4–5 lines)

Note: Individual reflections are personal to each team member and have not been pre-written on their behalf. Each student listed below should complete their own 4–5 line reflection covering: the most difficult engineering problem encountered and how it was solved; a key engineering decision made personally and the evidence behind it; new knowledge acquired independently; and what would be redesigned if the project were repeated.

Manoj M (192511060)

This is the central scheduling engine of PrintEase and is treated as the primary evidence of the system's DAA implementation throughout this chapter, since it directly exposes the data structures and their complexities. The module receives print jobs with their initial priority scores, continuously computes each job's dynamic priority using the aging relationship described in Section 3.5, and applies aging to increase the priority of jobs that have been waiting. It stores all pending jobs in a Binary Max-Heap, from which the job with the highest dynamic priority is selected first; after a job is processed the heap is re-heapified so that the invariant is restored for the remaining jobs.

REFERENCES

References are listed in IEEE style, in order of citation.

[1] M. Mustafa et al., “Cloud-based automated print queue management,” International Journal of Computer Applications, 2025.

[2] D. Dhabliya et al., “Cloud-based print management with secure printing,” ShodhKosh: Journal of Visual and Performing Arts, 2025.

[3] L. Li et al., “Print-job sequence optimization and scheduling,” IEEE Access, 2024.

[4] C. C. Antonio, “High Performance Computing for Big Data in Distributed Systems,” Distributed Processing Systems, vol. 1, no. 2, pp. 10–17, 2020, doi: 10.38007/DPS.2020.010202.

[5] E. Burova, S. Grishunin, S. Suloeva, and A. Stepanchuk, “The Cost Management of Innovative Products in an Industrial Enterprise Given the Risks in the Digital Economy,” International Journal of Technology, vol. 12, no. 7, pp. 1339–1348, 2021, doi: 10.14716/ijtech.v12i7.5333.

[6] H. Dan, “Exploring the Innovative Path of Human Resources Management of Construction Enterprises in the Internet Plus Era,” Modern Economics and Management, vol. 4, no. 1, pp. 63–65, 2023.

[7] M. A. N. S. U. R. Eshov, “Influence Assessment of Enterprise Management Value Based on Coefficients Methods Under the Risk Conditions,” Advances in Mathematics: Scientific Journal, vol. 9, no. 9, pp. 7573–7598, 2020, doi: 10.37418/amsj.9.9.104.

[8] J. J. Ferreira, J. M. Lopes, S. Gomes, and H. G. Rammal, “Industry 4.0 Implementation: Environmental and Social Sustainability in Manufacturing Multinational Enterprises,” Journal of Cleaner Production, vol. 404, p. 136841, 2023, doi: 10.1016/j.jclepro.2023.136841.

[9] OpenAI, “ChatGPT (2026 version) [Large language model],” 2026. [Online]. Available: https://chat.openai.com

[10] Anthropic, “Claude [Large language model],” 2026. [Online]. Available: https://claude.ai

APPENDICES

Appendix A – Detailed Calculations

Binary Max-Heap complexity (standard result, applied to this project's job queue of n pending jobs):

- Insertion of a new job: the job is placed at the next free leaf position and "sifted up" while its priority exceeds its parent's. The heap has height $\lceil \log_2(n+1) \rceil$, so at most O(log n) swaps are performed → Insert = O(log n).
- Extraction of the highest-priority job (heap root): the root is removed, the last leaf is moved to the root, and it is "sifted down" while a child has greater priority. This again costs at most O(log n) swaps → Extract-Max = O(log n).
- Building a heap from n unordered jobs (bulk load): repeated sifting from the last internal node upward is bounded by Σ(h/2^h) over all levels, giving Build-Heap = O(n), rather than the naive O(n log n) from n sequential insertions.
- Hash-table lookup by Token/Order ID: with a well-distributed hash function, average-case lookup, insertion and deletion are O(1); worst case (all keys colliding) degrades to O(n), which is why a good hash function and adequate table sizing are assumed.

Illustrative aging calculation (worked example for explanation only, not measured system data): let the base priority for an "Assignment" document be P0 = 50 points (Figure 4.2), and let the aging factor be k = 1 point per minute waited. A job that has waited w = 20 minutes would have a dynamic priority of P(t) = 50 + 1×20 = 70 points at that instant, which would place it above a freshly submitted "Others" document (P0 = 25) but still below a freshly submitted, urgent "Hall Ticket" request (P0 = 100 + 50 = 150, from the base 100 points plus the +50-point urgent bonus shown in Figure 4.2). This example illustrates the mechanism only; actual aging-factor values used in the deployed system are a configurable parameter (Section 3.3).

Appendix B – Engineering Drawings / Schematics

See Figure 3.1 (PrintEase System Architecture) for the complete system schematic, and Figures 4.1–4.5 for the module-level interface schematics referenced throughout Chapter 4.

Appendix C – Source Code / Code Repository Information

Only essential excerpts illustrating the core scheduling algorithm are reproduced below; the complete source code is maintained separately in the project's approved repository. The excerpt illustrates the Binary Max-Heap sift-up operation used during job insertion, consistent with the algorithm described in Sections 3.5 and Appendix A.

```
// Illustrative excerpt: Binary Max-Heap insertion (sift-up)
// heap[] stores PrintJob objects ordered by dynamicPriority
void PriorityHeap::insertJob(PrintJob job) {
    heap.push_back(job);              // place at next free leaf
    int i = heap.size() - 1;
    while (i > 0) {
        int parent = (i - 1) / 2;
        if (heap[parent].dynamicPriority >= heap[i].dynamicPriority)
            break;                    // heap property restored
        swap(heap[parent], heap[i]);  // sift up
        i = parent;
    }
}

// Illustrative excerpt: aging update applied to a waiting job
int computeDynamicPriority(int basePriority, int waitMinutes, int agingFactor) {
    return basePriority + (waitMinutes * agingFactor);
}
```

Appendix D – Datasheets (Student, Admin, Shopkeeper/Printer, Print Jobs)

Field-level record structure inferred from the module interfaces shown in Figures 4.1–4.5:

Student / User Record

Field	Description
Username	Chosen display name at registration (Figure 4.1)
Email Address	Contact/login identifier
Hostel	Selected hostel (used for the Admin “Orders by Hostel” breakdown, Figure 4.5)
Room Number	Room within the selected hostel
Password	Authentication credential (stored securely; not displayed in plaintext)
Role	Student / Shopkeeper (role toggle at registration, Figure 4.1)

Print Job Record

Field	Description
Token / Order ID	Unique identifier, e.g., PE-RVB71D (Figure 4.4)
Username	Submitting student's username
Hostel / Room	Delivery/collection location
Document Type	Hall Ticket / Exam Record / Assignment / Others (base priority, Figure 4.2)
Urgency Level	Normal / Urgent (priority bonus, Figure 4.2)
Print Type	Black & White / Colour
Print Side	Single Side / Double Side
Number of Copies	Copies requested
Base Priority (P0)	Computed from document type and urgency (Module 2)
Dynamic Priority P(t)	P0 plus aging contribution (Modules 2–3)
Status	Pending / Printing / Printed (Completed)

Field	Description
Submitted Timestamp	Date/time of submission (Figure 4.4)

Print-Shop / Printer Record

Field	Description
Shop Name	Partner print-shop identifier (Admin "Partner Shops" count, Figure 4.5)
Pending / Printing / Delivered Counts	Live job counts by status (Figure 4.3)
Total Orders	Cumulative orders received by the shop (Figure 4.3)

Admin Record

Field	Description
Total Students	Aggregate registered student count (Figure 4.5)
Partner Shops	Aggregate registered print-shop count (Figure 4.5)
Total / Active Orders	Platform-wide order counters (Figure 4.5)
Orders by Hostel	Distribution of orders across hostels (Figure 4.5)

Appendix E – Experimental / Raw Data

No controlled-experiment raw data set (e.g., repeated-trial waiting-time measurements against an FCFS baseline) was collected at the time of this report; the two corresponding test items are marked "Pending" in Table 4.2 and identified as future work in Section 5.2. The prototype-snapshot figures reported in Section 4.4 (Figures 4.3–4.5) are the only quantitative data captured to date.

Appendix F – Ethics / Institutional Approval

Not applicable. This is an academic software prototype involving no human-subject experimentation, and no institutional ethics approval was required or sought.

Appendix G – Project Meeting / Progress Records

To be maintained by the team as a project log (meeting dates, attendees, decisions); no such records were supplied as source material for this report.

Appendix H – User / Industry Feedback

Not collected at this stage of the project.

Appendix I – Publications / Patents / Competitions / Product Outcomes

None reported to date.

Appendix J – Individual Contribution Evidence

Mandatory for team projects. The table below mirrors the front-matter Individual Contribution Statement and is to be completed by the team with supporting evidence (e.g., commit history, module ownership) prior to submission.

Student	Specific Responsibilities	Evidence
Manoj M (192511060)	[To be finalised by team]	[To be finalised by team]
Theshon Mishmaan M (192572106)	[To be finalised by team]	[To be finalised by team]
Yokesh (192424174)	[To be finalised by team]	[To be finalised by team]
Jaya Krishna (192421259)	[To be finalised by team]	[To be finalised by team]
Dinesh (192424016)	[To be finalised by team]	[To be finalised by team]

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department/faculty assessor)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)